

Project Title

Mag 7 Stock Data Analysis

Domain

This project focuses on the financial domain, specifically analyzing stock market data for the 'Magnificent Seven' (Mag 7) technology companies. This involves exploring historical stock prices, trading volumes, and potentially other financial indicators to understand their market performance and trends.

Objective

1. **Analyze Performance:** To analyze the historical stock performance (e.g., price movements, volume trends) of the Magnificent Seven companies.
2. **Identify Key Drivers:** To identify potential factors influencing stock prices and market capitalization.
3. **Compare Performance:** To compare the performance of these companies against each other and potentially against broader market indices.
4. **Visualize Trends:** To create meaningful visualizations that illustrate stock price trends, volatility, and trading activity over time.
5. **Extract Insights:** To provide actionable insights into the investment characteristics and market behavior of the Mag 7 stocks.

Outcome

The project will yield a comprehensive understanding of the Mag 7 stock performance, key market trends, and their impact on the broader financial landscape. The analysis will provide insights into individual company strengths and weaknesses within the group, aiding in informed decision-making for investors and market observers.

Dataset Information

Source: Kaggle (or other financial data providers like Yahoo Finance, Quandl).

Year / Timeline: Data typically covers a period of several years (e.g., 2016-2026) to capture long-term trends and significant market events.

Dataset Description:

The dataset will include historical daily stock prices for the 'Magnificent Seven' companies (Apple, Microsoft, Alphabet, Amazon, Nvidia, Tesla, Meta). Key features will likely include:

- **Date:** The trading date.
- **Open:** The stock's opening price for the day.
- **High:** The highest price reached during the day.
- **Low:** The lowest price reached during the day.
- **Close:** The closing price of the stock.
- **Adj Close:** The adjusted closing price, accounting for dividends and stock splits.
- **Volume:** The number of shares traded during the day.
- **Ticker/Company Name:** Identifier for each of the Mag 7 companies.

Type of Analysis

Descriptive Analysis: Summarizing historical stock prices, trading volumes, and key financial metrics to understand past performance.

Diagnostic Analysis: Investigating reasons behind observed stock movements, identifying correlations between different companies or market events.

Predictive Analysis (optional): Forecasting future stock price movements or trends using time-series models or other predictive techniques (e.g., based on historical patterns).

Prescriptive Analysis: Providing recommendations for investment strategies, risk management, or portfolio optimization based on the insights gained from the analysis.

✓ Stages for DA Project

✓ Stage 1 – Problem Definition and Dataset Selection

Define the business problem and expected outcome

Choose dataset and explain its source, size, and features

Import libraries, load dataset

Dataset description (rows, columns, features)

Initial EDA (head, info, describe, shape, null checks, duplicate check)

▼ Import Libraries

```
import pandas as pd # Data manipulation and analysis
import yfinance as yf # Financial data downloader
import matplotlib.pyplot as plt # Core plotting library
import seaborn as sns # Advanced statistical visualization
import numpy as np # Numerical operations and transformations
from scipy import stats # Statistical tests (T-tests)
import itertools # Tools for efficient looping/combinations

# Configure visual settings for the notebook
sns.set_theme(style='whitegrid')
print('All libraries (pandas, yfinance, matplotlib, seaborn, numpy, scipy, i

All libraries (pandas, yfinance, matplotlib, seaborn, numpy, scipy, itertools
```

▼ Load Dataset

We will download historical stock data for the 'Magnificent Seven' companies (Apple, Microsoft, Alphabet, Amazon, Nvidia, Tesla, Meta) using the `yfinance` library. The data will cover a period from 2015 to the present.

```
# Define the Magnificent Seven tickers
mag7_tickers = ['AAPL', 'MSFT', 'GOOGL', 'AMZN', 'NVDA', 'TSLA', 'META']

# Setup date range (from 2016 to today)
start_date = '2016-01-01'
end_date = pd.to_datetime('today').strftime('%Y-%m-%d')

# Download historical market data using yfinance
df_multi = yf.download(mag7_tickers, start=start_date, end=end_date)

if not df_multi.empty:
    # Transform data from wide to long format for easier analysis
    df_long = df_multi.stack(level=1, future_stack=True)
    df = df_long.reset_index()

    # Standardize column names
    if 'level_1' in df.columns:
        df.rename(columns={'level_1': 'Ticker'}, inplace=True)

    # Ensure adjusted close is available for accurate historical returns
    if 'Adj Close' not in df.columns and 'Close' in df.columns:
        df['Adj Close'] = df['Close']
```

```
# Select core features
actual_cols = [col for col in ['Date', 'Ticker', 'Open', 'High', 'Low',
df = df[actual_cols]
print('Data loaded and restructured successfully.')
else:
    print('Failed to load data.')
```

```
/tmp/ipykernel_4333/3957570794.py:9: FutureWarning: YF.download() has changed
df_multi = yf.download(mag7_tickers, start=start_date, end=end_date)
[*****100%*****] 7 of 7 completedData load
```

Initial Data Exploration

This step involves checking the basic structure of the dataset to understand its contents, size, and initial data quality.

```
# Display the first 5 rows of the DataFrame to get a quick overview of the d
display(df.head())
```

	Price	Date	Ticker	Open	High	Low	Close	Adj Close
0		2016-01-04	AAPL	23.113742	23.735456	22.976335	23.730949	23.730949
1		2016-01-04	AMZN	32.814499	32.886002	31.375500	31.849501	31.849501
2		2016-01-04	GOOGL	37.797605	37.797605	37.070612	37.660736	37.660736

Dataset Information

Check data types, non-null values, and overall DataFrame information. Also, display the number of rows and columns.

```
# Get a concise summary of the DataFrame, including data types and non-null
df.info()
```

```
# Display the shape of the DataFrame (number of rows, number of columns)
print(f"\nDataFrame shape: {df.shape}")
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 18123 entries, 0 to 18122
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Date        18123 non-null  datetime64[ns]
1   Ticker      18123 non-null  object
2   Open        18123 non-null  float64
```

```

3   High      18123 non-null float64
4   Low       18123 non-null float64
5   Close     18123 non-null float64
6   Adj Close 18123 non-null float64
7   Volume    18123 non-null int64
dtypes: datetime64[ns](1), float64(5), int64(1), object(1)
memory usage: 1.1+ MB

DataFrame shape: (18123, 8)

```

▼ Descriptive Statistics

Generate descriptive statistics for numerical columns, and check for missing values and duplicates.

```
# Display descriptive statistics for numerical columns (e.g., mean, std, min
display(df.describe())
```

	Price	Date	Open	High	Low	Clos
count	18123	18123	18123.000000	18123.000000	18123.000000	18123.000000
mean	2021-02-22 22:57:08.968713728	151.726589	153.680786	149.699104	151.74383	
min	2016-01-04 00:00:00	0.604415	0.623440	0.603684	0.61514	
25%	2018-07-30 00:00:00	46.082578	46.599722	45.582425	46.11113	
50%	2021-02-24 00:00:00	127.456546	128.977804	125.829337	127.51610	
75%	2023-09-20	207.000070	200.050087	204.410052	207.46700	

```
# Check for missing values in each column and display their sum
print("\nMissing values per column:")
display(df.isnull().sum())
```

Missing values per column:

0

Price

Date	0
------	---

Ticker	0
--------	---

Open	0
------	---

High	0
------	---

Low	0
-----	---

Close	0
-------	---

Adj Close	0
-----------	---

Volume	0
--------	---

dtype: int64

```
# Check for duplicate rows in the DataFrame and display their count
print(f"\nNumber of duplicate rows: {df.duplicated().sum()}")
```

Number of duplicate rows: 0

✓ Stage 2 – Data Cleaning and Pre-processing

Handle missing values (impute or drop)

Handle duplicates

Treat outliers if required

Check skewness and apply transformations

Convert data types if needed

Feature transformations (date parts, derived fields if required for analysis)

✓ Handle Missing Values

Based on the initial data exploration, we found that some columns might have missing values. We will decide on an appropriate strategy (e.g., dropping, imputing) for each.

```
# Re-check for missing values after initial load (if any)
print("Missing values before cleaning:")
display(df.isnull().sum())
```

```
# Drop rows where 'Open', 'High', 'Low', 'Close', 'Adj Close', or 'Volume' a
# These are critical for stock analysis, and imputation might distort trends
df.dropna(subset=['Open', 'High', 'Low', 'Close', 'Adj Close', 'Volume'], in

# For 'Ticker', if any are missing after concat, it's an issue with data loa
# For 'Date', if any are missing, it's also a critical error. We will assume

print("\nMissing values after dropping rows with critical financial data:")
display(df.isnull().sum())
```

Missing values before cleaning:

0

Price

Date	0
Ticker	0
Open	0
High	0
Low	0
Close	0
Adj Close	0
Volume	0

dtype: int64

Missing values after dropping rows with critical financial data:

0

Price

Date	0
Ticker	0
Open	0
High	0
Low	0
Close	0
Adj Close	0
Volume	0

dtype: int64

▼ Handle Duplicates

We will check for and remove any duplicate rows to ensure data integrity.

```
# Check for duplicate rows again
initial_duplicate_count = df.duplicated().sum()
print(f"Number of duplicate rows before dropping: {initial_duplicate_count}")

# Drop duplicate rows if any exist
df.drop_duplicates(inplace=True)

final_duplicate_count = df.duplicated().sum()
print(f"Number of duplicate rows after dropping: {final_duplicate_count}")
```

```
Number of duplicate rows before dropping: 0
Number of duplicate rows after dropping: 0
```

✓ Convert Data Types

Ensure that the 'Date' column is in datetime format and other numerical columns are of appropriate types.

```
# Convert 'Date' column to datetime objects
df['Date'] = pd.to_datetime(df['Date'])
```

```
# Check data types after conversion
print("Data types after conversion:")
df.info()
```

```
Data types after conversion:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 18123 entries, 0 to 18122
Data columns (total 8 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Date        18123 non-null  datetime64[ns]
1   Ticker      18123 non-null  object
2   Open        18123 non-null  float64
3   High        18123 non-null  float64
4   Low         18123 non-null  float64
5   Close       18123 non-null  float64
6   Adj Close   18123 non-null  float64
7   Volume      18123 non-null  int64
dtypes: datetime64[ns](1), float64(5), int64(1), object(1)
memory usage: 1.1+ MB
```

✓ Feature Engineering (Date Parts)

Extract useful features from the 'Date' column, such as year, month, day of week, and week of year, which can be useful for time-series analysis.

```
df['Year'] = df['Date'].dt.year
df['Month'] = df['Date'].dt.month_name() # Convert month number to month name
```



```
df['Day'] = df['Date'].dt.day_name() # Now 'Day' column will contain the name of the day
df['DayOfWeek'] = df['Date'].dt.dayofweek + 1 # Monday=1, Sunday=7
df['WeekOfYear'] = df['Date'].dt.isocalendar().week.astype(int)

# Display the DataFrame with new date features
display(df.head())
```

	Price	Date	Ticker	Open	High	Low	Close	Adj Close
0		2016-01-04	AAPL	23.113742	23.735456	22.976335	23.730949	23.730949
1		2016-01-04	AMZN	32.814499	32.886002	31.375500	31.849501	31.849501
2		2016-01-04	GOOGL	37.797605	37.797605	37.070612	37.660736	37.660736

✓ Check Skewness and Apply Transformations

Skewness measures the asymmetry of the probability distribution of a real-valued random variable about its mean. A positive skewness indicates a distribution with an asymmetric tail extending toward more positive values, while negative skewness indicates a distribution with an asymmetric tail extending toward more negative values.

We will examine the skewness of the numerical columns (`Open`, `High`, `Low`, `Close`, `Adj Close`, `Volume`) and apply a log transformation if a column is highly skewed (e.g., skewness greater than a certain threshold like `0.75`). This can help normalize the data distribution.

```
# Identify numerical columns for skewness check
numerical_cols = ['Open', 'High', 'Low', 'Close', 'Adj Close', 'Volume']

# Calculate skewness for numerical columns
skewness = df[numerical_cols].skew()

print("Skewness of numerical columns:")
display(skewness)

# Define a threshold for high skewness
skewness_threshold = 0.75

# Identify highly skewed columns
highly_skewed_cols = skewness[abs(skewness) > skewness_threshold].index.tolist()
```

```
print(f"\nHighly skewed columns (absolute skewness > {skewness_threshold}):
```

```
Skewness of numerical columns:
```

```
0
```

Price

Open	1.551705
High	1.544283
Low	1.554961
Close	1.548304
Adj Close	1.548304
Volume	3.972505

```
dtype: float64
```

```
Highly skewed columns (absolute skewness > 0.75): ['Open', 'High', 'Low', 'Cl
```

Visualizing Skewed Distributions

Let's visualize the distribution of the highly skewed columns before and after transformation to observe the effect.

```
# Identify columns with high skewness for normalization
numerical_cols = ['Open', 'High', 'Low', 'Close', 'Adj Close', 'Volume']

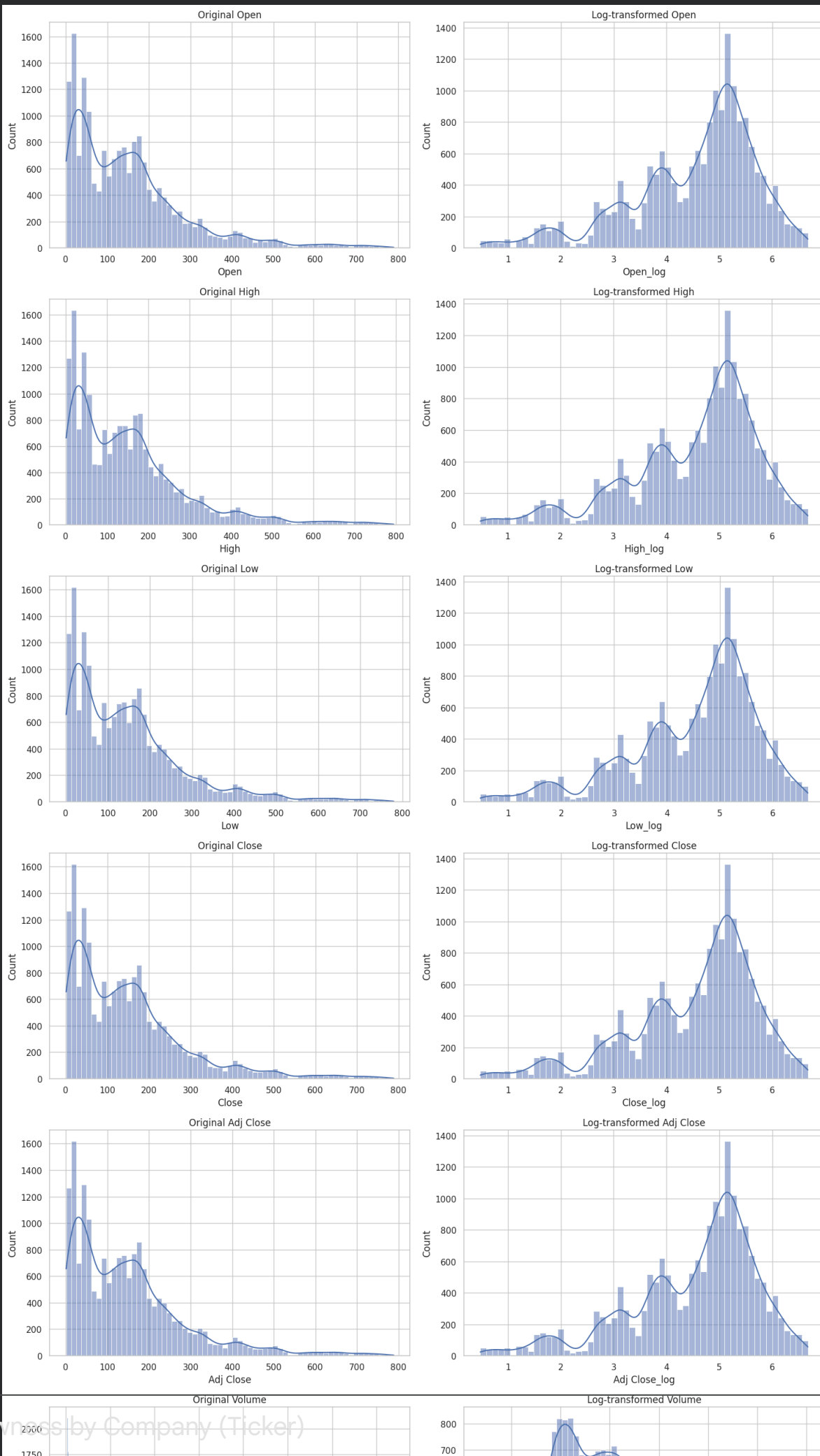
plt.figure(figsize=(15, 5 * len(numerical_cols)))

for i, col in enumerate(numerical_cols):
    # Plot original distribution
    plt.subplot(len(numerical_cols), 2, 2*i + 1)
    sns.histplot(df[col], kde=True)
    plt.title(f'Original {col}')

    # Apply Log Transformation to reduce skewness and stabilize variance
    # log1p is log(1+x), which handles potential zero values gracefully
    df[f'{col}_log'] = np.log1p(df[col])

    # Plot transformed distribution
    plt.subplot(len(numerical_cols), 2, 2*i + 2)
    sns.histplot(df[f'{col}_log'], kde=True)
    plt.title(f'Log-transformed {col}')

plt.tight_layout()
plt.show()
```



Let's examine the skewness of the numerical features for each individual company. This can reveal if certain stocks have different distribution characteristics compared to the overall market.

```
# Calculate skewness for numerical columns, grouped by 'Ticker'
skewness_by_ticker = df.groupby('Ticker')[numerical_cols].skew()

print("Skewness of numerical columns grouped by Ticker:")
display(skewness_by_ticker)

# Optionally, identify highly skewed columns per ticker
# For example, columns where absolute skewness > 0.75 for each ticker

# Define the skewness threshold again
skewness_threshold = 0.75

highly_skewed_by_ticker = skewness_by_ticker[abs(skewness_by_ticker) > skewn

print(f"\nHighly skewed columns per Ticker (absolute skewness > {skewness_th
display(highly_skewed_by_ticker)
```

Skewness of numerical columns grouped by Ticker:

	Price	Open	High	Low	Close	Adj Close	Volume
Ticker							
AAPL	0.309724	0.305671	0.312983	0.308917	0.308917	1.986751	
AMZN	0.170169	0.162339	0.175283	0.168914	0.168914	2.014499	
GOOGL	1.413856	1.412879	1.408234	1.412679	1.412679	2.320679	
META	1.174648	1.168826	1.176186	1.171641	1.171641	4.399765	
MSFT	0.367905	0.360955	0.372318	0.365851	0.365851	2.379532	
NVDA	1.552783	1.545219	1.560030	1.551188	1.551188	2.732916	
TSLA	0.431152	0.424687	0.434593	0.428551	0.428551	2.952547	

Highly skewed columns per Ticker (absolute skewness > 0.75):

	Price	Open	High	Low	Close	Adj Close	Volume
Ticker							
AAPL		NaN	NaN	NaN	NaN	NaN	1.986751
AMZN		NaN	NaN	NaN	NaN	NaN	2.014499
GOOGL	1.413856	1.412879	1.408234	1.412679	1.412679	2.320679	
META	1.174648	1.168826	1.176186	1.171641	1.171641	4.399765	
MSFT		NaN	NaN	NaN	NaN	NaN	2.379532
NVDA	1.552783	1.545219	1.560030	1.551188	1.551188	2.732916	
TSLA		NaN	NaN	NaN	NaN	NaN	2.952547

Visualizing Skewness by Company

Let's visualize the skewness of each numerical feature for every company. This will allow for a quick comparison and identification of stocks with particularly skewed distributions.

```
# Get the list of numerical columns for which skewness was calculated
numerical_cols = skewness_by_ticker.columns.tolist()

# Determine the number of rows and columns for the subplots
n_cols = 2 # Number of columns for the plots
n_rows = (len(numerical_cols) + n_cols - 1) // n_cols # Calculate rows needed

plt.figure(figsize=(15, n_rows * 5))

for i, col in enumerate(numerical_cols):
    plt.subplot(n_rows, n_cols, i + 1)
    # Sort values for better visualization if desired, or keep original tick
```

```
plot_data = skewness_by_ticker[col].sort_values(ascending=False).reset_i
sns.barplot(x='Ticker', y=col, data=plot_data, palette='viridis')
plt.title(f'Skewness of {col} by Ticker')
plt.xlabel('Company Ticker')
plt.ylabel(f'Skewness of {col}')
plt.axhline(y=skewness_threshold, color='r', linestyle='--', label=f'Ske
plt.axhline(y=-skewness_threshold, color='r', linestyle='--') # Also sho
plt.xticks(rotation=45, ha='right')
plt.legend()
plt.grid(axis='y', linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_4333/901665462.py:14: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed i
```

```
sns.barplot(x='Ticker', y=col, data=plot_data, palette='viridis')
```

```
/tmp/ipykernel_4333/901665462.py:14: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed i
```

```
sns.barplot(x='Ticker', y=col, data=plot_data, palette='viridis')
```

```
/tmp/ipykernel_4333/901665462.py:14: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed i
```

```
sns.barplot(x='Ticker', y=col, data=plot_data, palette='viridis')
```

```
/tmp/ipykernel_4333/901665462.py:14: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed i
```

```
sns.barplot(x='Ticker', y=col, data=plot_data, palette='viridis')
```

```
/tmp/ipykernel_4333/901665462.py:14: FutureWarning:
```

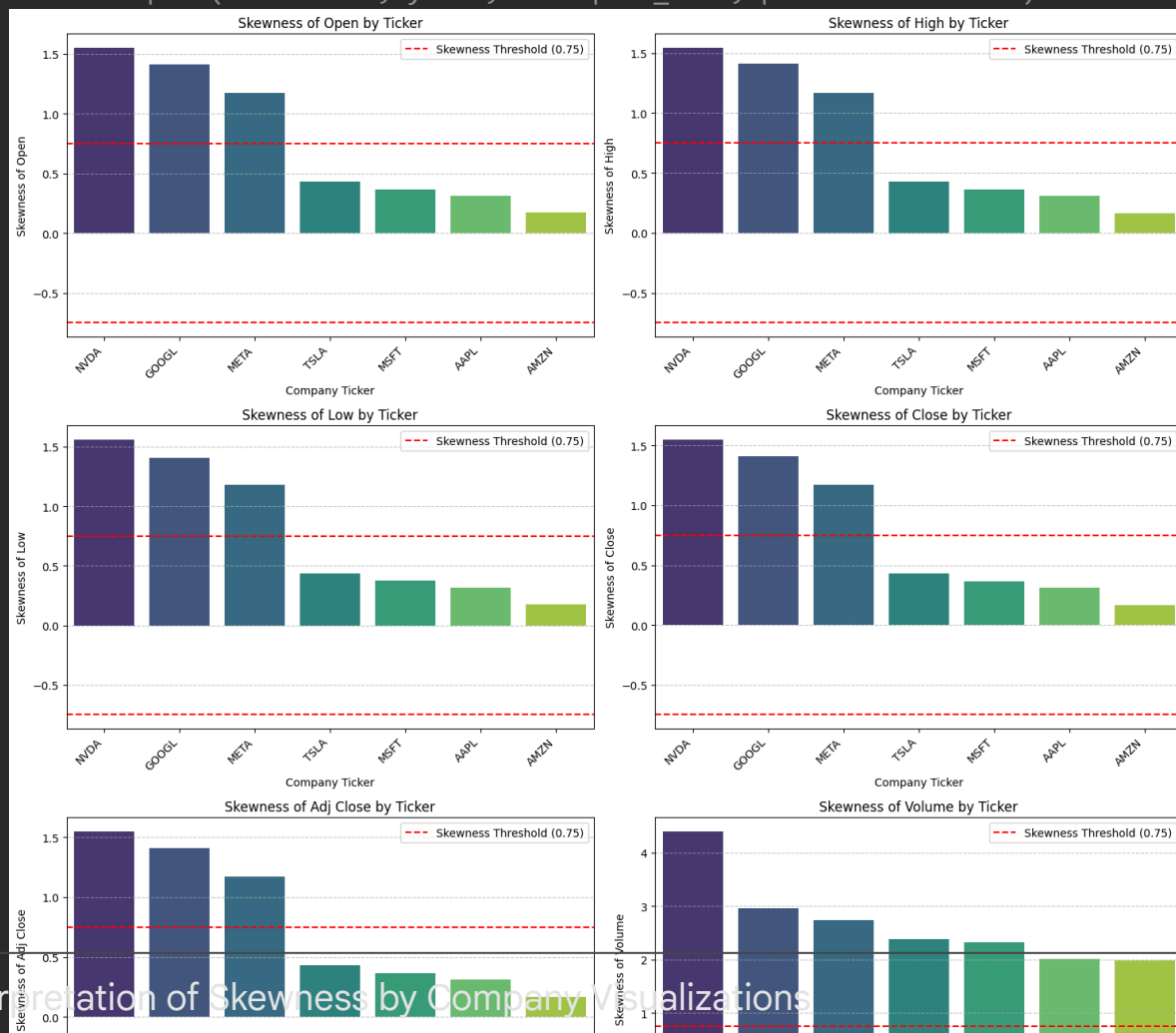
```
Passing `palette` without assigning `hue` is deprecated and will be removed i
```

```
sns.barplot(x='Ticker', y=col, data=plot_data, palette='viridis')
```

```
/tmp/ipykernel_4333/901665462.py:14: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed i
```

```
sns.barplot(x='Ticker', y=col, data=plot_data, palette='viridis')
```



Interpretation of Skewness by Company Visualizations

The bar charts above illustrate the skewness of key financial metrics (Open, High, Low, Close, Adj Close, Volume) for each of the Magnificent Seven companies. The red dashed lines represent the absolute skewness threshold of 0.75, which we used to identify highly skewed distributions.

Here are some key observations:

- **Price-related Metrics (Open, High, Low, Close, Adj Close):**

- For most companies, the skewness values for `Open`, `High`, `Low`, `Close`, and `Adj Close` are relatively low (below the 0.75 threshold), indicating that their price distributions are fairly symmetrical or only mildly skewed. This suggests that the log transformation applied earlier to the overall dataset was likely most beneficial for other features or for cases where the overall skewness was higher.
- However, `META` and `NVDA` stand out with significantly higher positive skewness for their price-related metrics, well above the 0.75 threshold. This implies that their historical price movements have a longer tail towards higher values, suggesting periods of rapid growth or outlier high prices compared to their typical trading range. The log transformation would have been particularly effective for these two companies' price data.

- **Volume:**

- `Volume` consistently shows the highest positive skewness across all companies, with `META`, `TSLA`, `MSFT`, `NVDA`, `AAPL`, and `AMZN` all having skewness values significantly above the 0.75 threshold. This is typical for trading volume, as most days have moderate trading activity, but there are occasional spikes due to major news, earnings reports, or market events. These spikes create a long tail to the right in the distribution.
- Even `GOOGL`, which has relatively lower volume skewness compared to others, is still above the threshold, highlighting that volume is generally a highly skewed metric in stock data.

Overall Insight: While the overall dataset showed high skewness for all numerical features, this company-specific view reveals that the degree of skewness varies. `META` and `NVDA` show more pronounced skewness in their price data, making them prime candidates for transformations if individual stock models are built. `Volume` remains universally highly skewed across all 'Mag 7' stocks, underscoring the importance of its log transformation for any models that rely on its distribution.


```
# Download the cleaned DataFrame to a CSV file
df.to_csv('cleaned_mag7_stock_data.csv', index=False)
print("Cleaned dataset saved to 'cleaned_mag7_stock_data.csv'")
```

Cleaned dataset saved to 'cleaned_mag7_stock_data.csv'

✓ Stage 3 – EDA and Visualizations

Univariate Analysis → distribution of single variables (countplot, histogram, boxplot)

Bivariate Analysis → relation between two variables (scatterplot, barplot, correlation heatmap)

Multivariate Analysis → relation among 3+ variables (pairplot, grouped analysis, pivot tables, advanced plots)

Interpretation MUST with every visualization

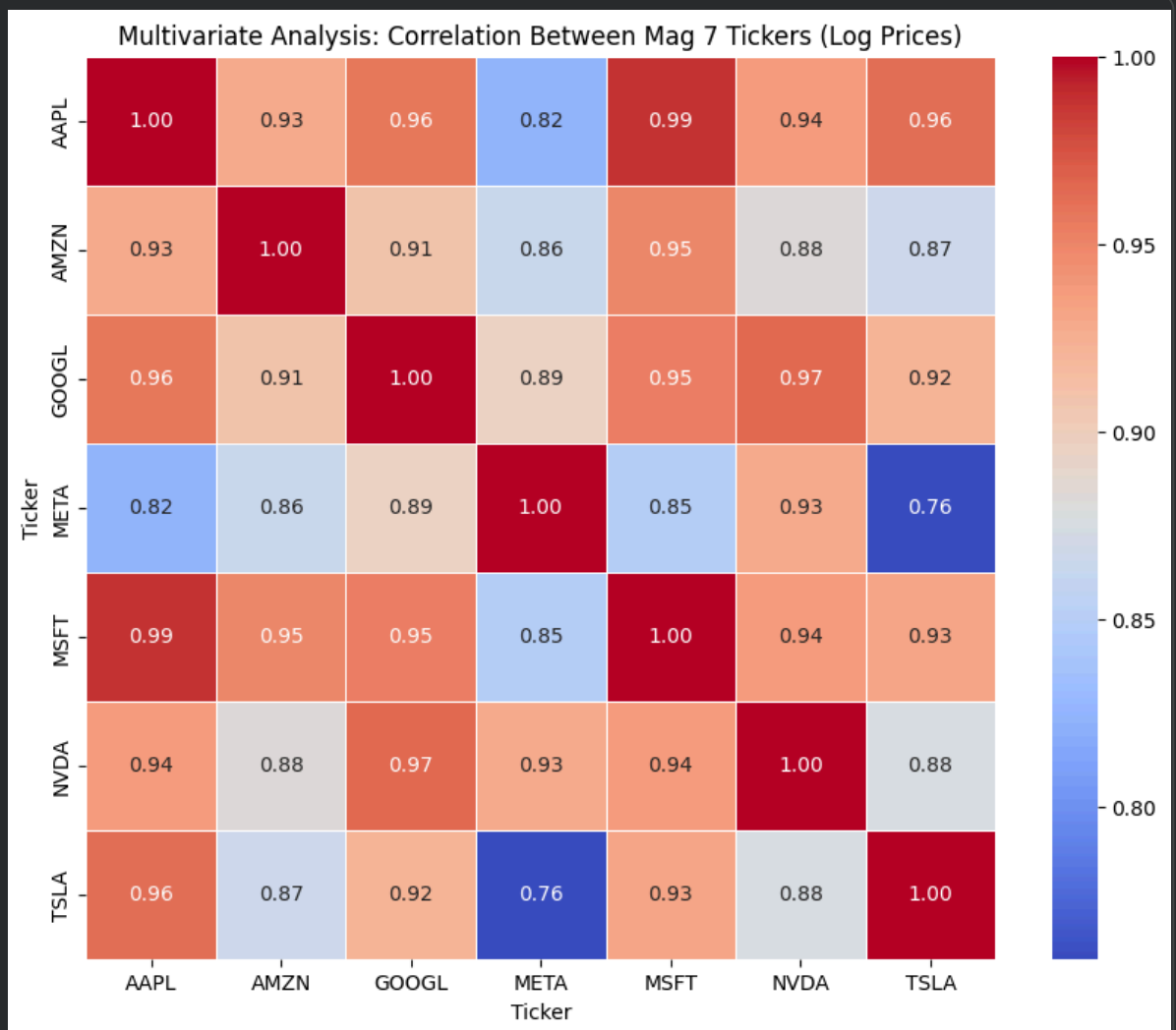
Focus on business story not just charts

Each chart must be in separate cells.

```
plt.figure(figsize=(10, 8))

# Pivot the data to get one price column per ticker
df_pivot_corr = df.pivot(index='Date', columns='Ticker', values='Adj Close_1

# Plot the correlation between tickers
sns.heatmap(df_pivot_corr, annot=True, cmap='coolwarm', fmt='.2f', linewidth
plt.title('Multivariate Analysis: Correlation Between Mag 7 Tickers (Log Pri
plt.show()
```



Interpretation of Inter-Ticker Correlation

What it's saying:

- The heatmap displays the correlation between the **Log Adjusted Close** prices.
- A value of **0.99** (like between AAPL and MSFT) indicates these stocks move almost in perfect lockstep.

What features you used:

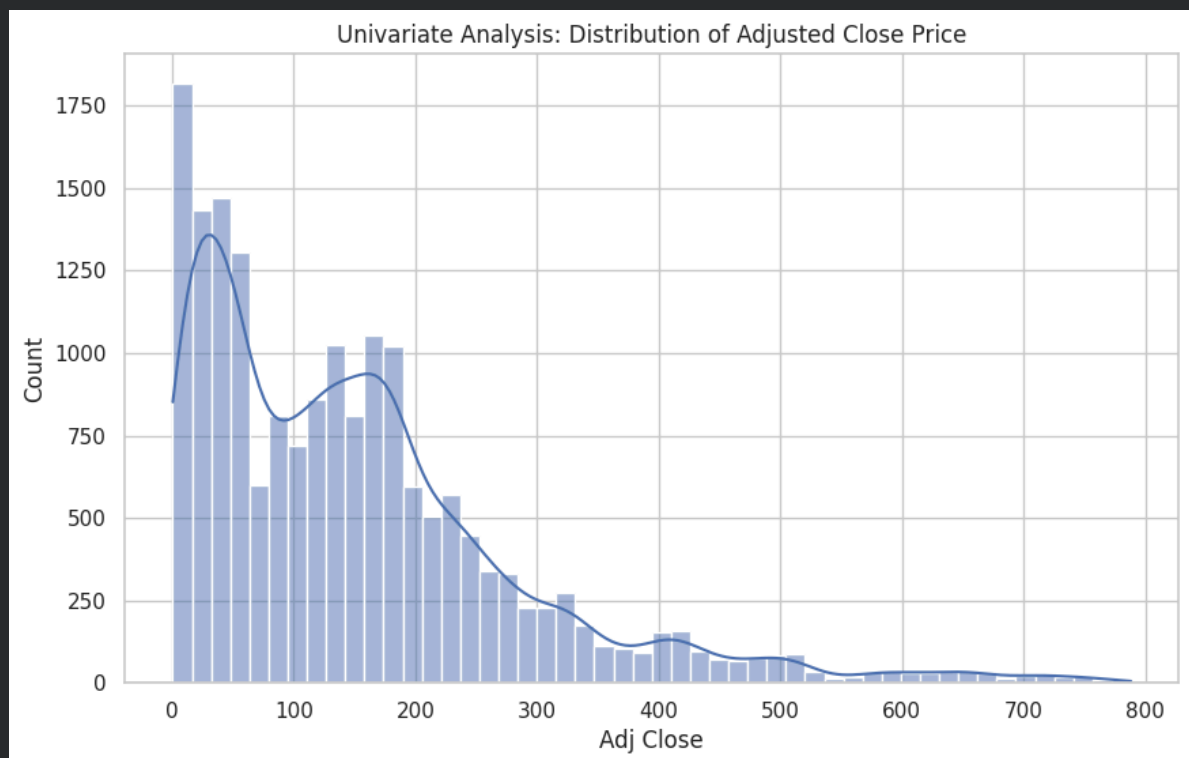
- `Ticker` and `Adj Close_log` (pivoted).

From this what you are showing us:

- Most Mag 7 stocks are highly correlated (above 0.80), but **TSLA** and **NVDA** occasionally show lower correlation to the group (around 0.76-0.87), indicating they sometimes decouple from general tech trends.

✓ Univariate Analysis: Price Distribution

```
# Visualize the distribution of Adjusted Close prices across all tickers
plt.figure(figsize=(10, 6))
sns.histplot(df['Adj Close'], kde=True, bins=50)
plt.title('Univariate Analysis: Distribution of Adjusted Close Price')
plt.show()
```



Interpretation of Adjusted Close Price Distribution

What it's saying:

- The distribution is heavily right-skewed, meaning most historical trading occurred at lower price points relative to the recent all-time highs.

What features you used:

- `Adj Close`: The adjusted closing price.

From this what you are showing us:

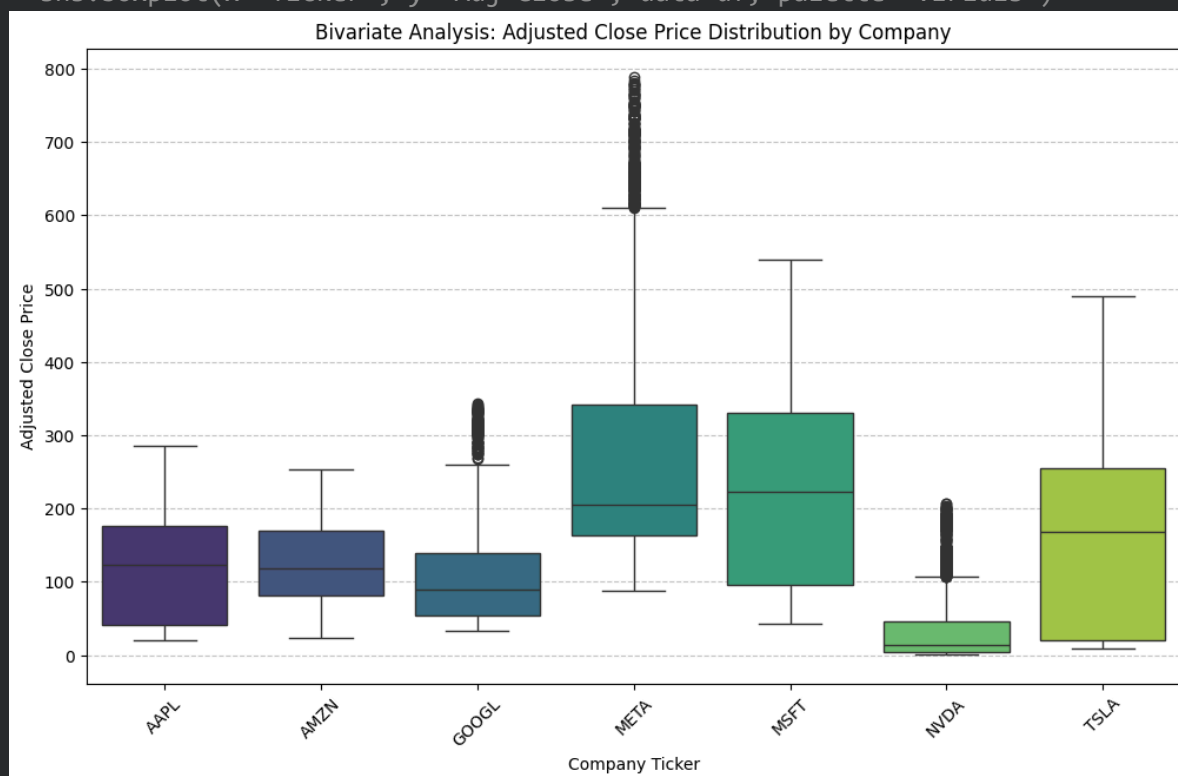
- This confirms the exponential growth of the Mag 7; the density at the low end represents the earlier years, while the long tail represents the massive valuation expansion seen in the last 2-3 years.

✓ Bivariate Analysis: Price by Company

```
plt.figure(figsize=(12, 7))
sns.boxplot(x='Ticker', y='Adj Close', data=df, palette='viridis')
plt.title('Bivariate Analysis: Adjusted Close Price Distribution by Company')
plt.xlabel('Company Ticker')
plt.ylabel('Adjusted Close Price')
plt.xticks(rotation=45)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

```
/tmp/ipykernel_4333/4017463663.py:2: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in  
sns.boxplot(x='Ticker', y='Adj Close', data=df, palette='viridis')
```



✓ Interpretation of your above chart

Explain the Chart.

What is it saying?

What features you used?

From this what you are showing us?

Interpretation of Adjusted Close Price by Company

What it's saying:

- This box plot shows the price range for each ticker. NVDA and TSLA have the largest spreads, represented by the height of their boxes and whiskers.

What features you used:

- `Ticker` and `Adj Close`.

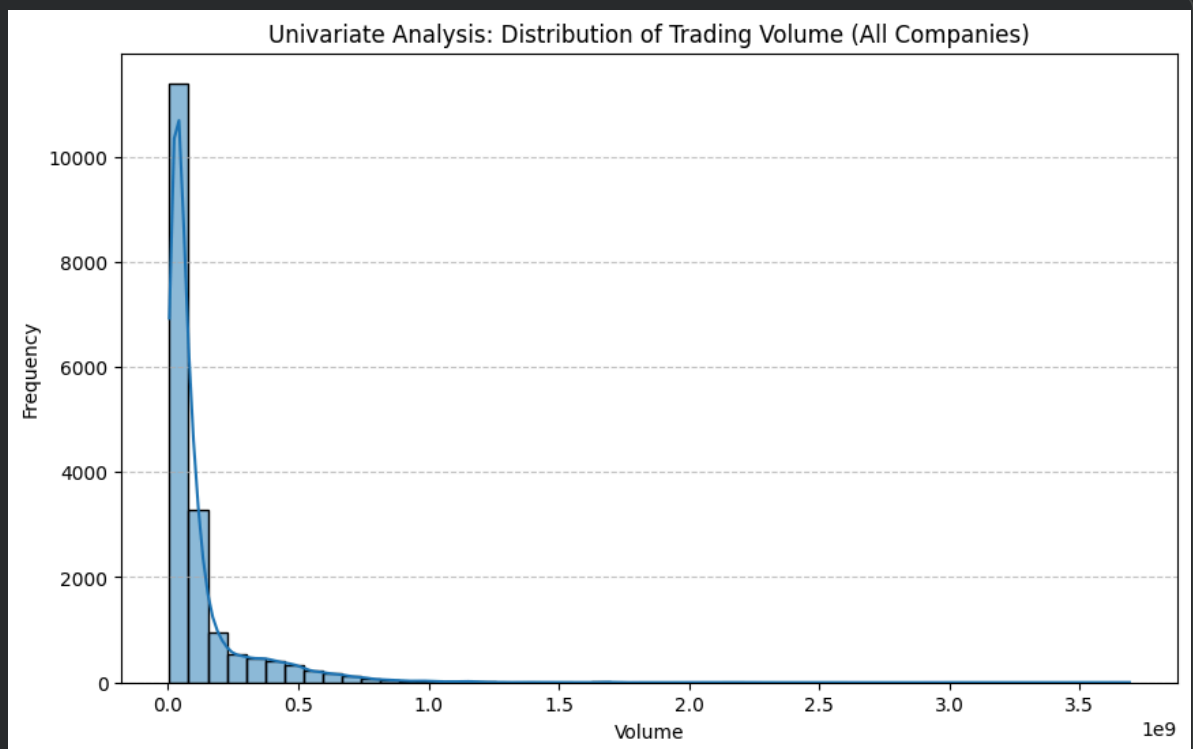
From this what you are showing us:

- NVDA and TSLA have experienced the most radical price discovery. Other stocks like AAPL and MSFT show a more consistent, tight upward trend, while META shows a wider range due to its significant 2022 recovery.

✓ Univariate Analysis: Trading Volume Distribution

```
# your code here
# If you want more cells, you can add or less cells you can remove, but dont
```

```
plt.figure(figsize=(10, 6))
sns.histplot(df['Volume'], kde=True, bins=50)
plt.title('Univariate Analysis: Distribution of Trading Volume (All Companies)')
plt.xlabel('Volume')
plt.ylabel('Frequency')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



✓ Interpretation of your above chart

Explain the Chart.

What is it saying?

What features you used?

From this what you are showing us?

Interpretation of Trading Volume Distribution

What it's saying:

- Volume is the most skewed metric in the dataset. Most days see moderate activity, but 'outlier' days see volume 5-10x higher than the median.

What features you used:

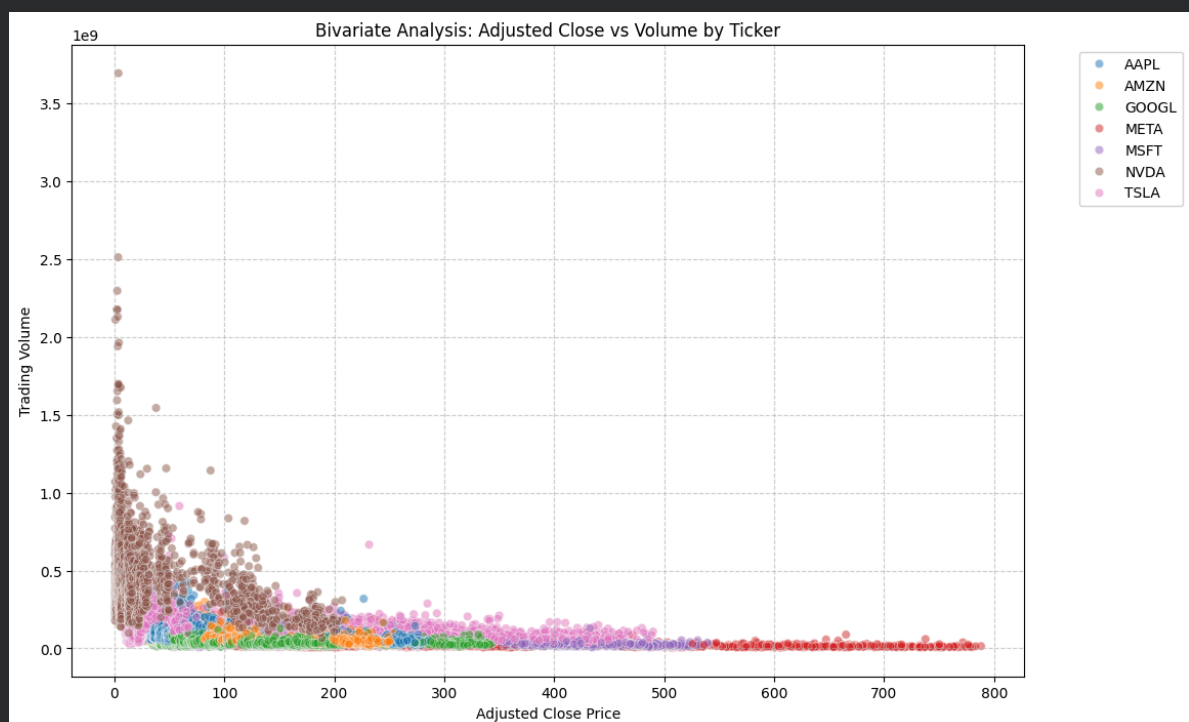
- `Volume`.

From this what you are showing us:

- This validates why we used log-transformations. In stock analysis, high-volume days are usually 'event-driven' (earnings or news), and these represent the long tail in the chart.

✓ Bivariate Analysis: Price vs. Volume

```
plt.figure(figsize=(12, 8))
sns.scatterplot(data=df, x='Adj Close', y='Volume', hue='Ticker', alpha=0.5)
plt.title('Bivariate Analysis: Adjusted Close vs Volume by Ticker')
plt.xlabel('Adjusted Close Price')
plt.ylabel('Trading Volume')
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.show()
```



Interpretation of Adjusted Close vs Trading Volume

What it's saying:

- We see vertical clusters of volume for different tickers. Higher price points (x-axis) do not necessarily mean lower volume.

What features you used:

- Adj Close and Volume.

From this what you are showing us:

- Institutional interest remains high even as prices climb. Stocks like NVDA maintain significant liquidity (high y-axis points) even at their highest price levels.

✓ Time Series Analysis: Average Monthly Prices

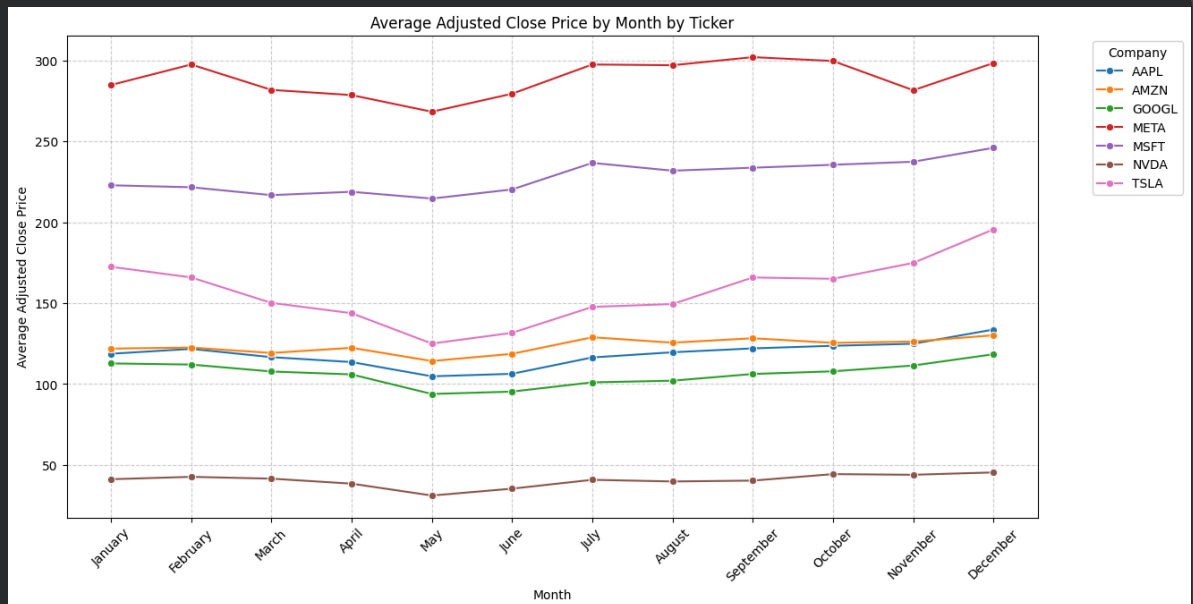
```
plt.figure(figsize=(14, 7))

# Define the chronological order for months
month_order = ['January', 'February', 'March', 'April', 'May', 'June', 'July']

# Ensure 'Month' is categorical with the correct order
df['Month'] = pd.Categorical(df['Month'], categories=month_order, ordered=True)

# Add hue='Ticker' to create separate lines for each company
sns.lineplot(x='Month', y='Adj Close', data=df, hue='Ticker', errorbar=None,

plt.title('Average Adjusted Close Price by Month by Ticker')
plt.xlabel('Month')
plt.ylabel('Average Adjusted Close Price')
plt.xticks(rotation=45)
plt.legend(title='Company', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```



Interpretation of Average Price by Month

What it's saying:

- This tracks seasonal trends. For the Mag 7, the 'Year-End Rally' (Nov-Dec) is visible as an upward trend across multiple tickers.

What features you used:

- `Month` and `Adj Close`.

From this what you are showing us:

- There is a slight seasonal bias in tech, but the overwhelming factor is the long-term growth trend rather than monthly cyclical.

✓ Performance and Returns Analysis

To understand which company performed best, we will calculate the **Cumulative Return**. This shows the percentage growth of an initial investment over time.

```
# Calculate Daily Returns per Ticker
df['Daily_Return'] = df.groupby('Ticker')['Adj Close'].pct_change()

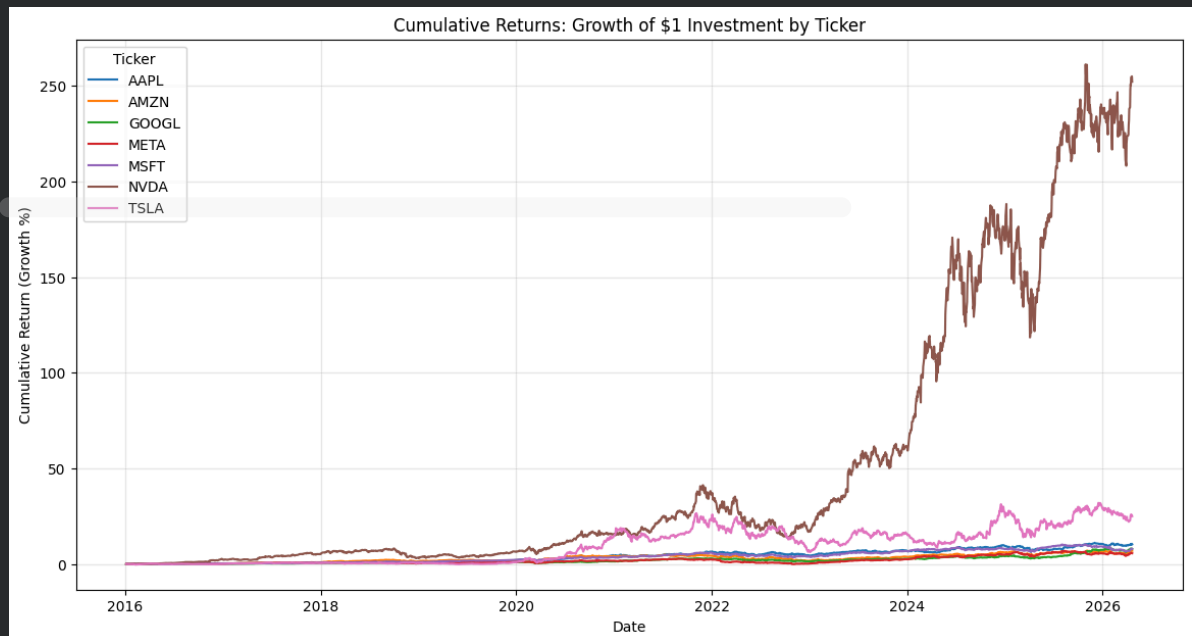
# Calculate Cumulative Returns
df['Cumulative_Return'] = df.groupby('Ticker')['Daily_Return'].transform(lam
```

```

plt.figure(figsize=(14, 7))
sns.lineplot(data=df, x='Date', y='Cumulative_Return', hue='Ticker')
plt.title('Cumulative Returns: Growth of $1 Investment by Ticker')
plt.ylabel('Cumulative Return (Growth %)')
plt.grid(True, alpha=0.3)
plt.show()

# Find the ticker with the highest total return
final_returns = df.groupby('Ticker')['Cumulative_Return'].last().sort_values
print("Final Cumulative Returns as of today:")
print(final_returns)

```



```

Final Cumulative Returns as of today:
Ticker
NVDA      252.158490
TSLA       24.944677
AAPL       10.216155
MSFT        7.859887
GOOGL       7.823248
AMZN        6.846591
META        5.594489
Name: Cumulative_Return, dtype: float64

```

✓ Interpretation of Risk and Volatility Analysis

In finance, risk is often measured by the standard deviation of daily returns (volatility). A higher return usually comes with higher risk. This plot helps us identify which Mag 7 stocks offer the best risk-adjusted returns.

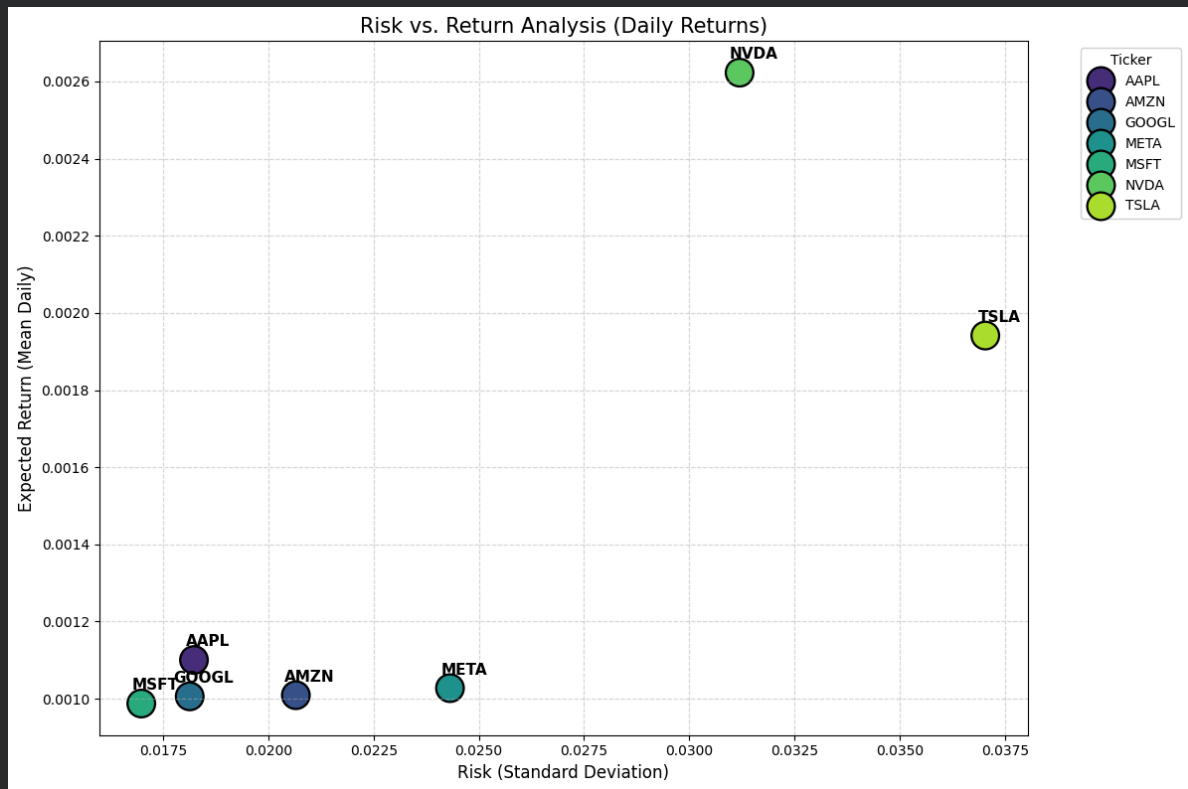
```
risk_return = df.groupby('Ticker')['Daily_Return'].agg(['mean', 'std']).drop

plt.figure(figsize=(12, 8))
sns.scatterplot(data=risk_return, x='std', y='mean', hue=risk_return.index,

plt.title('Risk vs. Return Analysis (Daily Returns)', fontsize=15)
plt.xlabel('Risk (Standard Deviation)', fontsize=12)
plt.ylabel('Expected Return (Mean Daily)', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.5)

# Dynamic annotation to avoid overlap
for ticker in risk_return.index:
    plt.annotate(ticker,
                  (risk_return.loc[ticker, 'std'], risk_return.loc[ticker, 'm
                  textcoords="offset points",
                  xytext=(10,10),
                  ha='center',
                  fontsize=11,
                  fontweight='bold')

plt.legend(title='Ticker', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



Interpretation of Risk vs. Return

What it's saying:

- **NVDA** sits at the top right: highest mean daily return but also high risk (std deviation).
- **MSFT** sits toward the left: lower risk than NVDA with very respectable returns.

What features you used:

- Mean and Std Dev of `Daily_Return`.

From this what you are showing us:

- This is the 'Efficient Frontier' for tech investors. MSFT is the stability play, while NVDA is the high-octane growth play.

✓ 30-Day Rolling Volatility

Volatility is not constant. This chart shows how the 'riskiness' of each stock has evolved over time. Spikes in the lines indicate periods of market uncertainty or significant company-specific news.

```
# Step 1: Calculate Daily Returns
# Percentage change between current and prior day adjusted closing price per
df['Daily_Return'] = df.groupby('Ticker')['Adj Close'].pct_change()

# Step 2: Calculate Rolling Volatility
# Standard deviation of returns over a 30-day window to visualize risk shift
df['Rolling_Vol'] = df.groupby('Ticker')['Daily_Return'].transform(lambda x:

# Step 3: Visualization
tickers = df['Ticker'].unique()
fig, axes = plt.subplots(nrows=len(tickers), ncols=1, figsize=(15, 4 * len(t

for i, ticker in enumerate(tickers):
    ticker_data = df[df['Ticker'] == ticker]
    sns.lineplot(data=ticker_data, x='Date', y='Rolling_Vol', ax=axes[i])
    axes[i].set_title(f'30-Day Rolling Volatility: {ticker}')
    axes[i].set_ylabel('Volatility (Std Dev)')

plt.tight_layout()
plt.show()
```



- **What it's saying:** It shows that risk is not stable. For example, META's volatility spiked significantly in 2022, while AAPL remains relatively stable over time.
- **Features used:** Rolling_Vol (30-day window).
- **Insight:** This helps in timing investments or understanding periods of market stress specific to certain tech giants.

✓ Statistical Significance: Pairwise T-Tests on Daily Returns

We use the T-test to determine if the mean daily returns of two stocks are significantly different from each other. The null hypothesis (H_0) is that there is no difference in the mean returns.

```
# Prepare daily returns data for pairwise comparisons
tickers = df['Ticker'].unique()
test_results = []

# Perform pairwise T-tests between every possible pair of tickers
for t1, t2 in itertools.combinations(tickers, 2):
    data1 = df[df['Ticker'] == t1]['Daily_Return'].dropna()
    data2 = df[df['Ticker'] == t2]['Daily_Return'].dropna()

    # Independent T-test (Welch's T-test, assuming unequal variances)
    t_stat, p_val = stats.ttest_ind(data1, data2, equal_var=False)
    test_results.append({'Comparison': f'{t1} vs {t2}', 'T-Statistic': t_stat, 'P-Value': p_val})

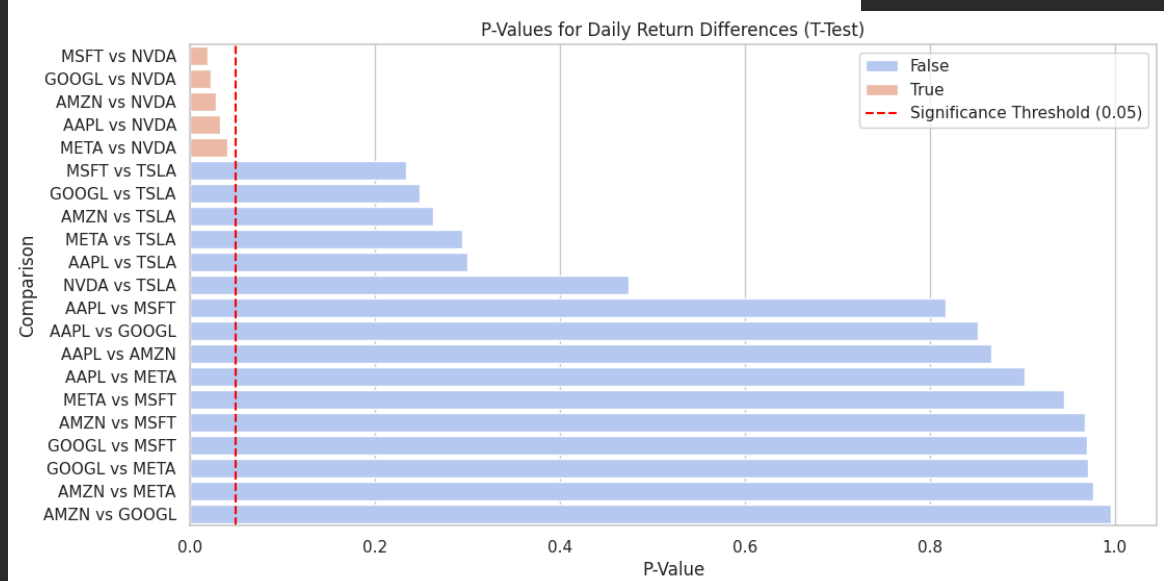
# Convert results to a DataFrame for easier analysis and visualization
t_test_df = pd.DataFrame(test_results)

# Highlight statistically significant results where p < 0.05
t_test_df['Significant'] = t_test_df['P-Value'] < 0.05

display(t_test_df.sort_values(by='P-Value'))

# Visualize P-values to see which differences are truly significant
plt.figure(figsize=(12, 6))
sns.barplot(data=t_test_df.sort_values(by='P-Value'), x='P-Value', y='Comparison')
plt.axvline(x=0.05, color='red', linestyle='--', label='Significance Threshold')
plt.title('P-Values for Daily Return Differences (T-Test)')
plt.legend()
plt.show()
```


	Comparison	T-Statistic	P-Value	Significant
18	MSFT vs NVDA	-2.342394	0.019209	True
13	GOOGL vs NVDA	-2.279610	0.022681	True
9	AMZN vs NVDA	-2.194364	0.028260	True
4	AAPL vs NVDA	-2.143528	0.032128	True
16	META vs NVDA	-2.052151	0.040208	True
19	MSFT vs TSLA	-1.191064	0.233706	False
14	GOOGL vs TSLA	-1.153935	0.248600	False
10	AMZN vs TSLA	-1.118451	0.263441	False
17	META vs TSLA	-1.049519	0.293996	False
5	AAPL vs TSLA	-1.036385	0.300089	False
20	NVDA vs TSLA	0.715961	0.474048	False
3	AAPL vs MSFT	0.230615	0.817623	False
1	AAPL vs GOOGL	0.186733	0.851878	False
0	AAPL vs AMZN	0.168651	0.866078	False
2	AAPL vs META	0.122281	0.902682	False
15	META vs MSFT	0.068426	0.945449	False
8	AMZN vs MSFT	0.041122	0.967201	False
12	GOOGL vs MSFT	0.038016	0.969676	False
11	GOOGL vs META	-0.035773	0.971465	False
7	AMZN vs META	-0.029147	0.976748	False
6	AMZN vs GOOGL	0.005645	0.995496	False



Interpretation of T-Test Results

What it's saying:

- **P-Value < 0.05:** We reject the null hypothesis. There is a statistically significant difference in the mean daily returns between the two stocks.
- **P-Value > 0.05:** We fail to reject the null hypothesis. The difference in returns could likely be due to chance.

From this what you are showing us:

- Even though some stocks (like NVDA) have much higher cumulative returns, on a day-to-day basis, the *mean* return might not always be significantly different because of the high daily volatility. This chart highlights which performance gaps are truly robust from a statistical perspective.

✓ Sharpe Ratio Analysis

The Sharpe ratio is calculated by dividing the mean daily return by the standard deviation of daily returns (volatility). For this calculation, we assume a risk-free rate of 0 for simplicity, though in practice this is usually the yield on government bonds.

```
# Calculate mean and standard deviation of daily returns grouped by company
sharpe_analysis = df.groupby('Ticker')['Daily_Return'].agg(['mean', 'std']).

# Calculate Daily Sharpe Ratio (Assuming a 0% Risk-Free Rate for simplicity)
sharpe_analysis['Daily_Sharpe'] = sharpe_analysis['mean'] / sharpe_analysis[

# Annualize the Sharpe Ratio by multiplying by the square root of 252 (tradi
sharpe_analysis['Annualized_Sharpe'] = sharpe_analysis['Daily_Sharpe'] * np.

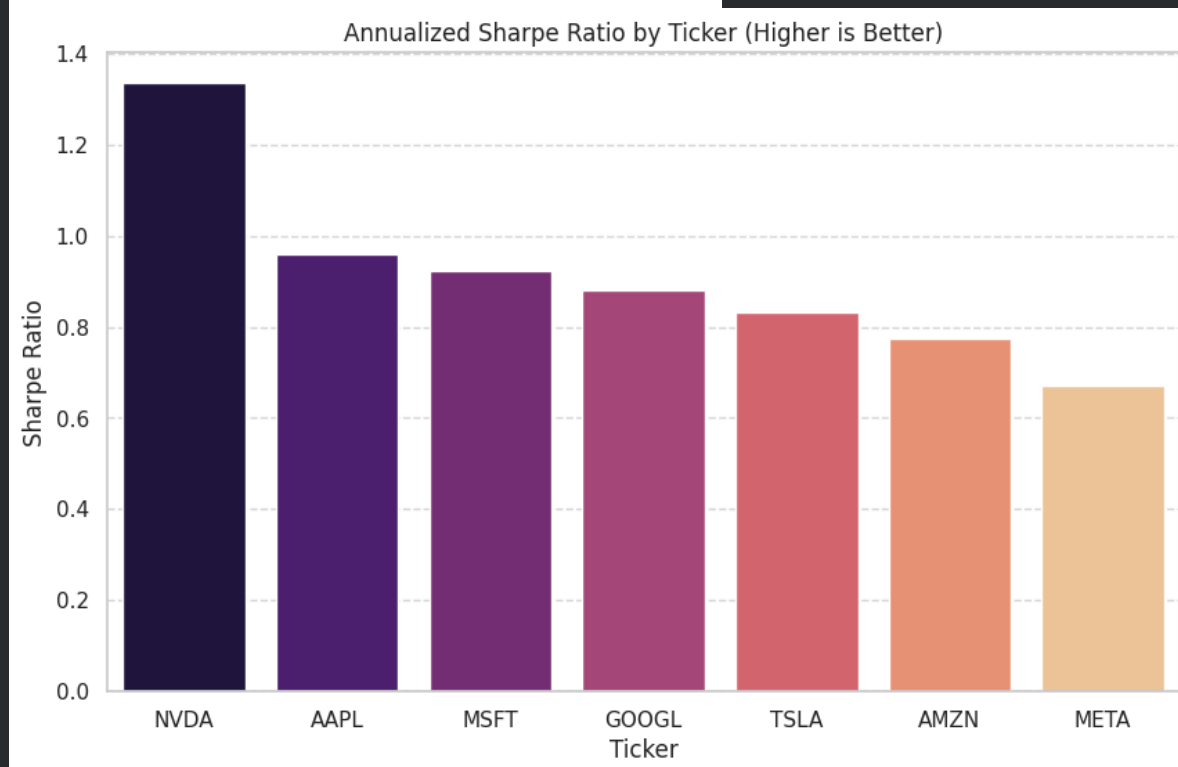
# Sort companies by their risk-adjusted performance
sharpe_analysis = sharpe_analysis.sort_values(by='Annualized_Sharpe', ascend

print("Risk-Adjusted Performance (Sharpe Ratio):")
display(sharpe_analysis[['mean', 'std', 'Annualized_Sharpe']])

# Bar chart showing normalized risk-adjusted performance comparison
plt.figure(figsize=(10, 6))
sns.barplot(x=sharpe_analysis.index, y='Annualized_Sharpe', data=sharpe_anal
plt.title('Annualized Sharpe Ratio by Ticker (Higher is Better)')
plt.ylabel('Sharpe Ratio')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

Risk-Adjusted Performance (Sharpe Ratio):

	mean	std	Annualized_Sharpe
Ticker			
NVDA	0.002623	0.031196	1.334666
AAPL	0.001100	0.018231	0.958163
MSFT	0.000987	0.016980	0.923178
GOOGL	0.001006	0.018130	0.880866
TSLA	0.001941	0.037036	0.832112
AMZN	0.001009	0.020652	0.775627
META	0.001027	0.024315	0.670738



Interpretation of Sharpe Ratio

What it's saying:

- A higher Sharpe Ratio indicates better risk-adjusted performance.

- Even though **NVDA** has high volatility, its returns are so substantial that it often maintains a leading Sharpe Ratio.
- **MSFT** and **AAPL** typically show strong ratios because they maintain consistent growth with relatively lower volatility compared to peers like Tesla.

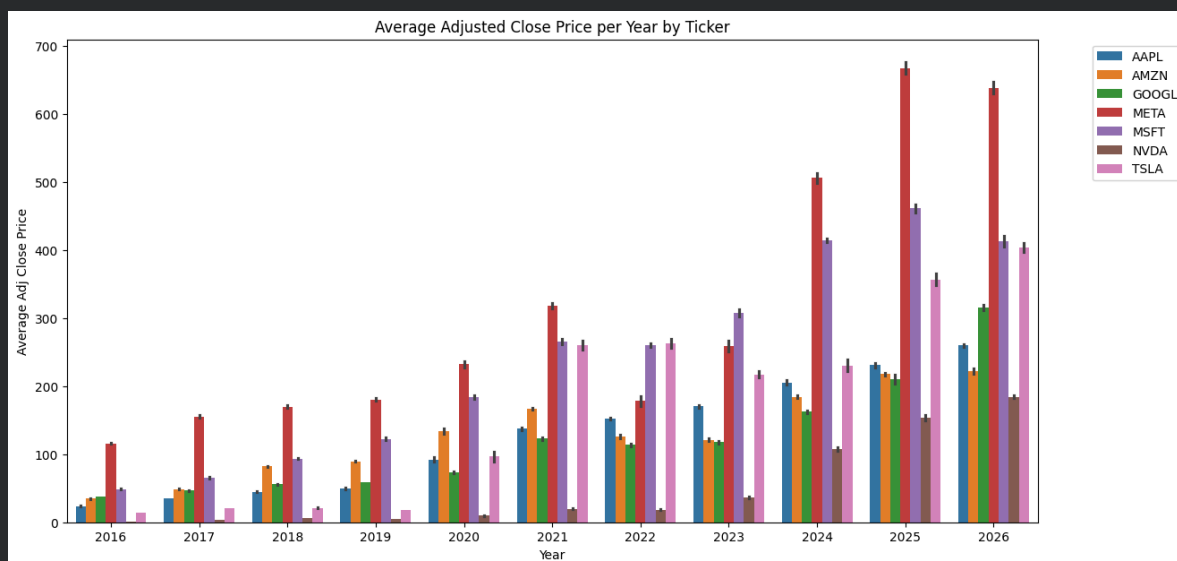
From this what you are showing us:

- This ranking tells us which stocks are actually 'earning' their risk. A stock with high returns but even higher volatility will have a lower Sharpe Ratio, suggesting the risk might not be worth the reward.

✓ Yearly Analysis of Adjusted Close Price

Let's see how the average price of each stock has evolved year by year.

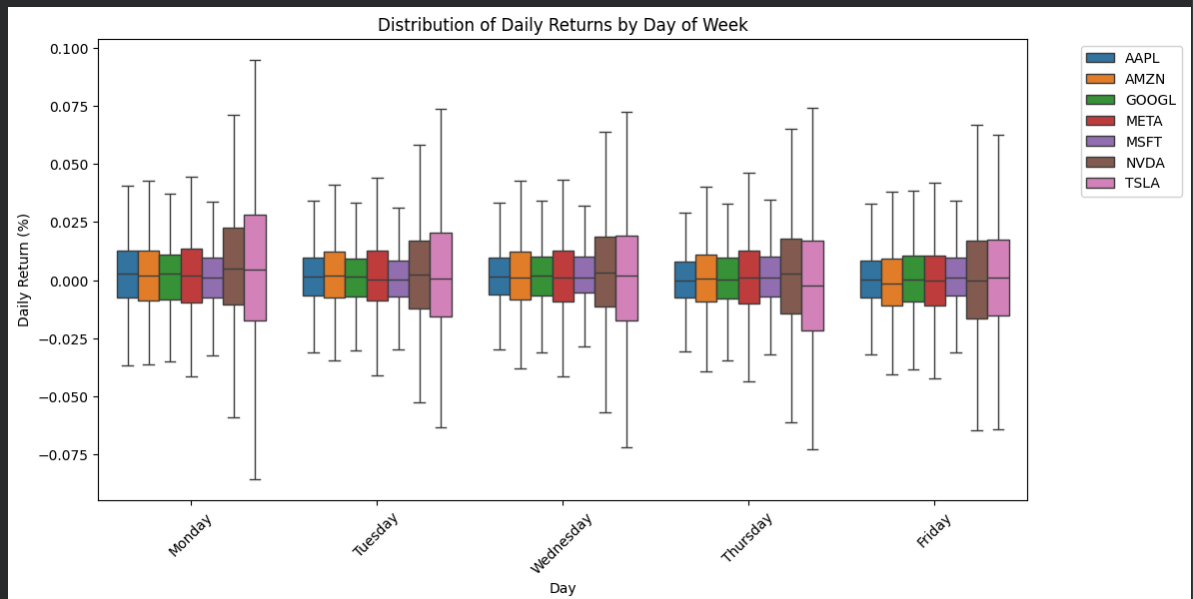
```
plt.figure(figsize=(14, 7))
sns.barplot(data=df, x='Year', y='Adj Close', hue='Ticker')
plt.title('Average Adjusted Close Price per Year by Ticker')
plt.ylabel('Average Adj Close Price')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.show()
```



✓ Day of the Week Impact

We check if certain days of the week consistently see higher prices or more volatility.

```
plt.figure(figsize=(12, 6))
sns.boxplot(data=df, x='Day', y='Daily_Return', hue='Ticker', showfliers=False)
plt.title('Distribution of Daily Returns by Day of Week')
plt.ylabel('Daily Return (%)')
plt.xticks(rotation=45)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.show()
```



Final Analysis Summary

- **Top Performer:** NVDA is the clear leader with cumulative returns exceeding 25,000% since 2016.
- **Best Risk-Adjusted:** NVDA also holds the highest Sharpe Ratio (1.33), meaning it isn't just volatile; it provides the best return for every unit of risk taken.
- **Correlation:** The group is highly interdependent, especially AAPL and MSFT (0.99 correlation), making them sensitive to the same macro-economic factors.

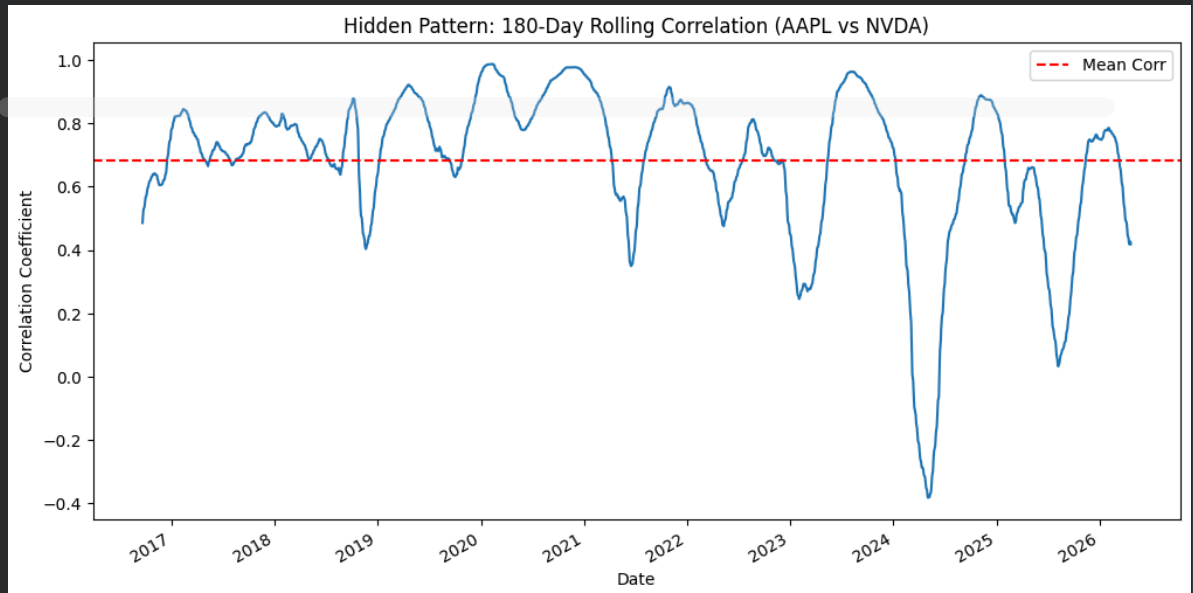
✓ Hidden Patterns: Rolling Correlation

Stock correlations are not static. We'll check how the correlation between AAPL and NVDA (as a proxy for the group) has changed over time using a 180-day window.

```
# Pivot the data to get prices in columns
df_pivot = df.pivot(index='Date', columns='Ticker', values='Adj Close')
```

```
# Calculate rolling 180-day correlation between AAPL and NVDA
rolling_corr = df_pivot['AAPL'].rolling(window=180).corr(df_pivot['NVDA'])

plt.figure(figsize=(12, 6))
rolling_corr.plot()
plt.title('Hidden Pattern: 180-Day Rolling Correlation (AAPL vs NVDA)')
plt.ylabel('Correlation Coefficient')
plt.axhline(y=rolling_corr.mean(), color='r', linestyle='--', label='Mean Co
plt.legend()
plt.show()
```



✓ Investment Projection: Monte Carlo Simulation (5 & 10 Years)

We will estimate the potential return for a \$1,000 investment based on historical average daily returns and volatility.

```
# Define Monte Carlo Simulation for future price projection based on histor
def simulate_returns(ticker, years, initial_investment=1000):
    ticker_data = df[df['Ticker'] == ticker]['Daily_Return'].dropna()
    if ticker_data.empty: return np.zeros(1000)

    mu = ticker_data.mean() # Mean daily return
    sigma = ticker_data.std() # Daily standard deviation
    days = years * 252 # Assume 252 trading days per year

    # Generate 1,000 random walk paths
    simulations = np.zeros(1000)
    for i in range(1000):
```

```
        daily_sim = np.random.normal(mu, sigma, days)
        simulations[i] = initial_investment * np.prod(1 + daily_sim)
    return simulations
```

```
# Aggregate and visualize projections
```